Nivel 10 — Modelos Locales de IA y Personalización / Fine-Tuning

Duración: 20 horas académicas | **Prácticas:** 5 | **Módulos:** 2 (10h cada uno)

Objetivo del nivel: ejecutar modelos de IA de forma local y personalizarlos mediante fine-tuning para casos de uso específicos.

Resultados de aprendizaje. Al finalizar, el participante será capaz de:

- Justificar técnicamente cuándo conviene un modelo local frente a una API en la nube.
- Instalar, ejecutar y consumir modelos open-source con Ollama y LM Studio.
- Estimar requisitos de hardware y elegir un nivel de cuantización adecuado.
- Decidir entre prompting, RAG y fine-tuning según el problema.
- Construir un dataset de calidad y ajustar un modelo con LoRA/QLoRA o con la API de OpenAI.
- Evaluar de forma objetiva el modelo ajustado frente al modelo base y considerar los aspectos éticos y de costos.

MÓDULO 10.1 — Modelos Locales de IA (10 h)

10.1.1 Por qué ejecutar modelos de IA localmente: privacidad, costos y control

Teoría. Un *modelo local* es un modelo de lenguaje (LLM) cuyos pesos se descargan y se ejecutan en hardware propio (laptop, estación de trabajo, servidor on-premise o VPS dedicado), sin enviar datos a un proveedor externo. Las tres razones principales son:

1. Privacidad y cumplimiento. Los datos nunca salen de la infraestructura de la organización. Esto es decisivo con información de misión crítica, datos personales, historias clínicas, información financiera, código propietario o documentos legales, y facilita el cumplimiento de políticas internas y de normativas de protección de datos (por ejemplo GDPR o leyes locales de datos personales).

2. Costos. Las APIs cobran por token: el costo crece linealmente con el uso. Un modelo local tiene costo fijo (hardware + electricidad + mantenimiento) y costo marginal casi nulo por consulta. Resulta rentable con volúmenes altos y constantes, y poco rentable con uso esporádico.

3. Control. Se controla la versión exacta del modelo (sin cambios silenciosos del proveedor), la disponibilidad (funciona sin internet), la latencia, los parámetros de generación, el *system prompt* y la posibilidad de personalizar el modelo con fine-tuning.

Contrapartidas que el profesional debe conocer: menor capacidad que los modelos de frontera más grandes, necesidad de hardware con memoria suficiente, mantenimiento y actualización a cargo del equipo, y responsabilidad propia sobre la seguridad.

Ejemplo. Una clínica quiere resumir historias médicas. Enviarlas a una API externa puede violar su política de confidencialidad. Con un modelo de 8B de parámetros corriendo en un servidor interno, los resúmenes se generan sin que ningún dato abandone la red local.

Matriz de decisión rápida:

Criterio	Favorece local	Favorece nube
Datos sensibles o regulados	Sí	No (salvo acuerdos específicos)
Volumen de consultas	Alto y estable	Bajo o variable
Calidad máxima de razonamiento	Limitada	Superior
Conectividad	Sin internet o restringida	Estable
Equipo técnico para mantener	Disponible	No disponible
Necesidad de personalizar pesos	Sí	Limitada

10.1.2 Modelos open-source: Llama, Mistral y otros modelos descargables

Teoría. Conviene distinguir tres conceptos:

- **Open-source estricto:** pesos, código y licencia permisiva (por ejemplo, Apache 2.0).
- **Open-weights:** los pesos son descargables, pero la licencia puede tener restricciones de uso (caso de varias familias de Meta).
- **Propietario:** solo accesible por API.

Familias principales (los nombres de versiones cambian con rapidez; verifique siempre la tarjeta del modelo y su licencia vigente):

Familia	Origen	Características
Llama	Meta	Muy extendida, gran ecosistema, tamaños desde 1B hasta más de 70B, buen desempeño general

Familia	Origen	Características
Mistral / Mixtral	Mistral AI (Francia)	Eficientes, varias con licencia Apache 2.0; Mixtral usa arquitectura *Mixture of Experts*
Gemma	Google	Modelos compactos y eficientes, buenos para equipos modestos
Qwen	Alibaba	Fuertes en multilingüe y código
Phi	Microsoft	Modelos pequeños (SLM) con muy buen rendimiento por tamaño
DeepSeek	DeepSeek	Modelos de razonamiento y código

Conceptos clave para elegir un modelo:

- **Parámetros (1B, 3B, 8B, 70B):** más parámetros suelen implicar más capacidad y más consumo de memoria.
- **Ventana de contexto:** cuántos tokens procesa a la vez (4K, 8K, 32K, 128K). Más contexto consume más memoria.
- **Variante *base* vs. *instruct*:** el modelo base solo predice texto; el *instruct* está ajustado para seguir instrucciones y conversar. Para uso práctico se parte casi siempre de *instruct*.
- **Modelos especializados:** código, visión, embeddings, razonamiento.
- **Idioma:** verifique el desempeño en español; algunos modelos pequeños rinden notablemente peor fuera del inglés.
- **Licencia:** revise si permite uso comercial y fine-tuning.

Dónde se descargan: Hugging Face (repositorio principal de modelos), la biblioteca de Ollama y el catálogo integrado de LM Studio.

Regla práctica de selección: comience por el modelo más pequeño que resuelva su tarea con calidad aceptable. Pruebe con 3 a 5 casos reales de su negocio, no con benchmarks genéricos.

10.1.3 Herramientas para ejecución local: Ollama y LM Studio

Ollama. Herramienta de línea de comandos (con servicio en segundo plano) para descargar y ejecutar modelos con un solo comando. Disponible en Windows, macOS y Linux. Expone una **API REST local** en `http://localhost:11434`, y además una ruta compatible con el formato de OpenAI, lo que permite reutilizar código escrito para esa API.

Comandos esenciales:

```
ollama pull llama3.2          # descarga el modelo
ollama run llama3.2           # chat interactivo en terminal
ollama list                   # modelos instalados
ollama show llama3.2          # detalles (parámetros, cuantización, contexto)
ollama ps                     # modelos cargados en memoria
ollama rm llama3.2            # eliminar un modelo
```

Consumo por API:

```
curl http://localhost:11434/api/generate -d '{
  "model": "llama3.2",
  "prompt": "Explica qué es la cuantización en dos frases.",
  "stream": false
}'
```

Personalización ligera con un **Modelfile** (no es fine-tuning; solo fija comportamiento):

```
FROM llama3.2
PARAMETER temperature 0.3
PARAMETER num_ctx 4096
SYSTEM "Eres un asistente de soporte técnico. Responde siempre en español, de forma breve y precisa."
```

```
ollama create soporte-es -f Modelfile
ollama run soporte-es
```

LM Studio. Aplicación de escritorio con interfaz gráfica: buscador de modelos de Hugging Face, chat integrado, control visual de parámetros (temperatura, contexto, descarga a GPU) y un **servidor local compatible con la API de OpenAI**. Es ideal para quien se inicia, para comparar modelos rápidamente y para equipos sin experiencia en terminal.

Comparativa:

Aspecto	Ollama	LM Studio
Interfaz	Terminal y API	Gráfica
Automatización y scripts	Excelente	Buena (servidor local)
Facilidad inicial	Media	Alta
Despliegue en servidor	Muy adecuado	Orientado a escritorio
Gestión de modelos	`pull` / `run`	Catálogo visual

Otras alternativas: **llama.cpp** (motor base de muchas herramientas), **vLLM** (servicio de alto rendimiento en producción con GPU) y **Open WebUI** (interfaz web de chat sobre Ollama).

Ejemplo en Python (usando la biblioteca oficial de Ollama):

```
import ollama

respuesta = ollama.chat(
    model="llama3.2",
    messages=[{"role": "user", "content": "Resume en 3 puntos los beneficios
de un modelo local."}],
)
print(respuesta["message"]["content"])
```

10.1.4 Requisitos de hardware y optimización de modelos (cuantización)

Teoría. El recurso crítico al ejecutar un LLM es la **memoria** donde residen los pesos: la VRAM de la GPU o la RAM del sistema. La velocidad depende del ancho de banda de memoria y del cómputo disponible.

Estimación de memoria. Una regla útil:

> Memoria aproximada (GB) ≈ parámetros (en miles de millones) × bytes por parámetro, más un 10–20 % adicional para contexto y *caché KV*.

Bytes por parámetro según la precisión: FP16 ≈ 2 bytes; 8 bits ≈ 1 byte; 4 bits ≈ 0,5 bytes.

Modelo	FP16	8 bits	4 bits
3B	~6 GB	~3 GB	~1,5–2 GB
8B	~16 GB	~8 GB	~4,5–5 GB
13B	~26 GB	~13 GB	~7–8 GB
70B	~140 GB	~70 GB	~38–40 GB

Valores orientativos; el consumo real depende del formato y del tamaño de contexto.

Cuantización. Técnica que reduce la precisión numérica de los pesos (de 16 bits a 8, 5, 4 o menos) para disminuir memoria y acelerar la inferencia, a cambio de una pérdida de calidad generalmente pequeña en niveles moderados.

- **Formato GGUF:** estándar de llama.cpp, usado por Ollama y LM Studio. Los nombres indican el nivel: `Q8_0`, `Q5_K_M`, `Q4_K_M`, `Q3_K_S`, etc.

- **Q4_K_M:** suele ser el mejor equilibrio entre tamaño y calidad.
- **Q8_0:** casi sin pérdida perceptible, el doble de tamaño que Q4.
- **Por debajo de 4 bits:** la degradación se vuelve notable, sobre todo en modelos pequeños.
- Otros métodos: GPTQ y AWQ (orientados a GPU), y bitsandbytes (4 y 8 bits, base de QLoRA).

Perfiles de hardware orientativos:

Equipo	Qué puede ejecutar cómodamente
Laptop 8 GB RAM, sin GPU dedicada	Modelos de 1B–3B cuantizados
PC 16 GB RAM o GPU con 8 GB VRAM	Modelos de 7B–8B en 4 bits
GPU con 16–24 GB VRAM	Modelos de 13B–34B en 4 bits
Servidor con 48 GB o más de VRAM	Modelos de 70B cuantizados
Mac con Apple Silicon y memoria unificada	Según memoria total; muy eficiente para 7B–70B

Otras técnicas de optimización:

- **Descarga parcial a GPU (*offloading*):** repartir capas entre GPU y CPU cuando el modelo no cabe completo.
- **Reducir la ventana de contexto** (`num_ctx`) para ahorrar memoria.
- **Modelos más pequeños y especializados** en lugar de uno grande genérico.
- **Procesamiento por lotes y caché** de prompts repetidos.
- **Mixture of Experts:** activa solo parte de los parámetros por token.

Ejemplo. Un modelo de 8B en FP16 requiere cerca de 16 GB y no cabe en una GPU de 8 GB. Cuantizado a Q4_K_M ocupa unos 5 GB y corre fluido, con una pérdida de calidad menor para la mayoría de tareas de oficina.

10.1.5 Comparativa entre modelos locales y APIs en la nube

Teoría. No existe una opción universalmente mejor; se compara en seis dimensiones:

Dimensión	Modelo local	API en la nube
Calidad / capacidad	Buena en modelos medianos; inferior a los de frontera	Máxima disponible

Dimensión	Modelo local	API en la nube
Privacidad	Total (datos en casa)	Depende del contrato y del proveedor
Costo	Inversión inicial; costo marginal bajo	Sin inversión; pago por token
Latencia	Predecible, sin red; depende del hardware	Depende de red y de la carga del proveedor
Escalabilidad	Limitada por el hardware	Prácticamente ilimitada
Mantenimiento	A cargo del equipo	A cargo del proveedor
Disponibilidad offline	Sí	No
Personalización de pesos	Total	Limitada (según el proveedor)

Análisis de punto de equilibrio (simplificado). Sea H el costo mensual de hardware amortizado más energía y mantenimiento, y c el costo mensual estimado en API para el mismo volumen. Si $c > H$ de forma sostenida, el modelo local es económicamente competitivo, siempre que la calidad alcance.

Arquitectura híbrida (recomendada en muchos casos). Enrutar según la tarea:

- Tareas con datos sensibles o repetitivas y de baja complejidad → modelo local.
- Tareas que requieren razonamiento complejo con datos no sensibles → API en la nube.

Caso de estudio. Una consultora procesa 50.000 correos al mes para clasificarlos por prioridad. Con un modelo local de 8B ajustado a su criterio, el costo marginal es casi cero y los correos no salen de su red. Para redactar propuestas complejas a clientes, usa una API de frontera con información ya anonimizada.

PRÁCTICAS DEL MÓDULO 10.1

PRÁCTICA 1 — Instalación y ejecución de un modelo local con Ollama (≈ 4 h)

Objetivo: instalar Ollama, descargar un modelo, interactuar con él por terminal y por API, y crear una versión personalizada con Modelfile.

Requisitos: equipo con al menos 8 GB de RAM (ideal 16 GB), 10 GB de disco libre, Python 3.10+ (opcional para la parte de scripts).

Pasos:

1. **Instalación.** Descargue Ollama desde su sitio oficial según su sistema operativo (en Linux, mediante el script de instalación indicado en la web oficial). Verifique con `ollama --version`.
2. **Descarga del primer modelo.** Ejecute `ollama pull llama3.2` (o un modelo equivalente de 3B–8B que su equipo soporte).
3. **Chat en terminal.** Ejecute `ollama run llama3.2` y haga cinco preguntas: una factual, una de resumen de texto, una de redacción de correo en español, una de lógica simple y una de código.
4. **Inspección.** Use `ollama show llama3.2` y anote: número de parámetros, cuantización, longitud de contexto. Use `ollama ps` mientras el modelo está activo y anote la memoria usada.
5. **Consumo por API.** Ejecute el `curl` de la sección 10.1.3. Luego escriba un script de Python con la biblioteca `ollama` que reciba una pregunta y devuelva la respuesta.
6. **Personalización con Modelfile.** Cree el modelo `soporte-es` con un *system prompt* propio de su dominio (por ejemplo, un asistente para el área de RRHH o de soporte) y compruebe que responde con el tono definido.
7. **Segundo modelo.** Descargue un modelo de otra familia (por ejemplo Mistral o Gemma) y repita tres de las preguntas del paso 3.

Entregable: informe breve (1–2 páginas) con capturas de pantalla, tabla de datos de cada modelo (parámetros, cuantización, memoria, velocidad aproximada en tokens/s si está disponible mediante `ollama run --verbose`), el Modelfile y el script de Python.

Rúbrica (20 puntos): instalación y ejecución correctas (5), consumo por API (5), Modelfile funcional (5), análisis de observaciones (5).

PRÁCTICA 2 — Comparativa de rendimiento entre un modelo local y una API en la nube (≈ 6 h)

Objetivo: medir de forma reproducible calidad, latencia y costo de un modelo local frente a una API en la nube, con tareas reales.

Diseño del experimento:

1. **Defina 4 tareas** representativas: (a) resumen de un texto de 500 palabras; (b) clasificación de 20 mensajes de clientes en categorías; (c) extracción de datos en formato JSON a partir de un texto; (d) pregunta de razonamiento con varios pasos.
2. **Prepare 10 casos de prueba** (en total) con respuesta de referencia esperada.
3. **Modelos a comparar:** un modelo local vía Ollama y un modelo de una API en la nube (use la que tenga disponible).
4. **Parámetros iguales:** misma temperatura (por ejemplo 0,2) y mismo *prompt* en ambos.

Script de medición (esquema en Python):

```
import time, csv, ollama

casos = [
    {"id": 1, "tarea": "resumen", "prompt": "Resume en 3 líneas: ..."},
    # ... más casos
]

def consultar_local(prompt):
    t0 = time.time()
    r = ollama.chat(model="llama3.2",
                    messages=[{"role": "user", "content": prompt}],
                    options={"temperature": 0.2})
    return r["message"]["content"], time.time() - t0

def consultar_nube(prompt):
    # Reemplace por el SDK del proveedor elegido.
    # Devuelva (texto, segundos).
    ...

with open("resultados.csv", "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f)
    w.writerow(["id", "tarea", "modelo", "segundos", "respuesta"])
    for c in casos:
        for nombre, fn in (("local", consultar_local), ("nube",
consultar_nube)):
            texto, seg = fn(c["prompt"])
            w.writerow([c["id"], c["tarea"], nombre, round(seg, 2), texto])
```

Evaluación:

- **Calidad:** califique cada respuesta de 1 a 5 con una rúbrica escrita de antemano (exactitud, formato, claridad, idioma). Idealmente dos evaluadores, y sin saber qué modelo generó cada respuesta (evaluación ciega).
- **Latencia:** promedio y máximo en segundos.
- **Costo:** para la nube, tokens consumidos × precio publicado por el proveedor (consulte su tabla de precios vigente); para el local, estime hardware amortizado y energía.
- **Privacidad:** indique qué datos habría enviado a la nube.

Entregable: tabla comparativa, gráfico de calidad vs. latencia, y una **recomendación argumentada** (local, nube o híbrido) para un caso de negocio concreto.

Rúbrica (20 puntos): diseño del experimento (5), medición correcta (5), análisis de resultados (5), recomendación argumentada (5).

MÓDULO 10.2 — Fine-Tuning y Personalización (10 h)

10.2.1 Qué es el fine-tuning y cuándo aplicarlo frente a RAG o prompting

Teoría. El *fine-tuning* (ajuste fino) consiste en continuar el entrenamiento de un modelo ya preentrenado con un conjunto de datos propio y relativamente pequeño, modificando sus pesos (o añadiendo pesos adicionales) para que adopte un estilo, formato, vocabulario o comportamiento específico.

Idea central: el fine-tuning enseña *cómo* responder (estilo, formato, tarea); RAG aporta *qué información* usar (conocimiento actualizable). Es un error frecuente intentar "enseñar datos" al modelo con fine-tuning cuando lo adecuado es RAG.

Escalera de personalización (siempre en este orden):

1. **Prompting:** instrucciones claras, ejemplos (*few-shot*), formato de salida. Costo mínimo, iteración inmediata.
2. **RAG:** recuperar documentos propios y dárselos al modelo como contexto. Ideal para conocimiento cambiante y para citar fuentes.
3. **Fine-tuning:** cuando lo anterior no alcanza en consistencia, formato, tono o costo.
4. **Combinación:** un modelo ajustado para el estilo/tarea más RAG para el conocimiento.

Necesidad	Mejor herramienta
Respuestas con información actualizada o de documentos internos	RAG
Citar la fuente de la respuesta	RAG
Tono de marca, estilo de redacción consistente	Fine-tuning
Salida siempre en un formato estricto (JSON, plantilla)	Fine-tuning (o prompting con ejemplos si basta)
Clasificación especializada de alto volumen	Fine-tuning de un modelo pequeño
Reducir costo/latencia sustituyendo un modelo grande por uno pequeño ajustado	Fine-tuning
Tarea resuelta bien con buenas instrucciones	Prompting
Datos que cambian cada semana	RAG

Cuándo NO hacer fine-tuning: si no ha agotado el prompting; si no tiene un dataset de calidad; si el conocimiento cambia con frecuencia; si no puede medir el resultado.

Tipos de fine-tuning:

- **Completo (*full fine-tuning*):** actualiza todos los pesos. Muy costoso en memoria y cómputo.
- **Eficiente en parámetros (PEFT):** entrena solo una pequeña fracción de parámetros. **LoRA** y **QLoRA** son los más usados.
- **Ajuste supervisado (SFT):** aprende de pares instrucción-respuesta.
- **Alineación por preferencias (DPO, RLHF):** aprende de pares "respuesta preferida / rechazada". Es un nivel avanzado.

Ejemplo de decisión. Una empresa quiere un asistente que responda sobre su catálogo de productos (cambia cada mes) con el tono formal de la marca. Solución: **RAG** para el catálogo + instrucciones de tono en el *system prompt*; solo si el tono no se logra de forma consistente, se añade un ajuste fino ligero para el estilo.

10.2.2 Preparación de un dataset de entrenamiento para fine-tuning

Teoría. La calidad del dataset determina la calidad del resultado. Pocos ejemplos excelentes superan a muchos ejemplos mediocres.

Cantidad orientativa: para tareas de estilo o formato, desde unas 50 a 100 ejemplos de alta calidad puede notarse una mejora; lo habitual es trabajar con cientos o pocos miles. Verifique siempre los mínimos y límites que indique la herramienta o proveedor.

Formatos comunes:

Formato de chat (JSONL), usado por OpenAI y muchas bibliotecas; una línea por ejemplo:

```
{"messages": [{"role": "system", "content": "Eres el asistente de soporte de Acme. Responde en español, tono cordial y breve."}, {"role": "user", "content": "¿Cómo restablezco mi contraseña?"}, {"role": "assistant", "content": "Con gusto. Ve a Configuración > Seguridad > Restablecer contraseña y sigue los pasos que te enviaremos por correo."}]}
```

Formato instrucción-entrada-salida (Alpaca), usado en muchos flujos de LoRA:

```
{"instruction": "Clasifica el mensaje según su prioridad.", "input": "El sistema de pagos está caído desde hace una hora.", "output": "ALTA"}
```

Proceso de construcción del dataset:

1. **Definir la tarea con precisión:** "clasificar correos en 4 categorías" es mejor que "mejorar el asistente".
2. **Recolectar datos reales:** conversaciones, tickets, correos, plantillas, siempre con autorización.

3. **Anonimizar:** eliminar nombres, cédulas, teléfonos, direcciones, números de cuenta y cualquier dato personal o confidencial.
4. **Redactar o curar las respuestas ideales:** deben ser el estándar que usted desea que el modelo reproduzca.
5. **Limpiar:** eliminar duplicados, ejemplos contradictorios, errores ortográficos no deseados y textos vacíos.
6. **Balancear:** que las categorías o tipos de casos estén representados de forma razonable.
7. **Incluir casos difíciles y bordes:** preguntas fuera de alcance, entradas ambiguas, casos en que el modelo debe negarse o pedir aclaración.
8. **Dividir en conjuntos:** entrenamiento (~80 %), validación (~10 %) y prueba (~10 %). **El conjunto de prueba no se toca durante el entrenamiento.**
9. **Validar el formato** con un script antes de entrenar.

Datos sintéticos. Es posible usar un modelo más potente para generar ejemplos de entrenamiento, pero requieren revisión humana y conviene revisar los términos de uso del modelo generador respecto al entrenamiento de otros modelos.

Script de validación y división (Python):

```
import json, random

random.seed(42)
ejemplos = []
with open("dataset_completo.jsonl", encoding="utf-8") as f:
    for n, linea in enumerate(f, 1):
        d = json.loads(linea)
        msgs = d["messages"]
        assert msgs[-1]["role"] == "assistant", f"Línea {n}: falta respuesta"
        assert all(m["content"].strip() for m in msgs), f"Línea {n}:
contenido vacío"
        ejemplos.append(d)

random.shuffle(ejemplos)
n = len(ejemplos)
cortes = {"train": ejemplos[: int(n*0.8)],
          "val":   ejemplos[int(n*0.8): int(n*0.9)],
          "test":  ejemplos[int(n*0.9):]}

for nombre, datos in cortes.items():
```

```
    with open(f"{nombre}.jsonl", "w", encoding="utf-8") as f:
        for d in datos:
            f.write(json.dumps(d, ensure_ascii=False) + "\n")
    print(nombre, len(datos))
```

Errores frecuentes: datos con información personal sin anonimizar; ejemplos del conjunto de prueba filtrados al de entrenamiento; respuestas inconsistentes entre sí; dataset demasiado homogéneo que provoca sobreajuste.

10.2.3 Fine-tuning con herramientas accesibles (OpenAI fine-tuning, LoRA)

A) Fine-tuning mediante API (ejemplo: OpenAI)

Es la vía más sencilla: se sube el archivo JSONL, la plataforma entrena y devuelve un identificador de modelo propio que se consume como cualquier otro. No requiere GPU propia. Las desventajas: el modelo queda alojado en el proveedor (no hay pesos descargables), el costo se cobra por tokens de entrenamiento y de uso, y los datos se envían al proveedor.

```
from openai import OpenAI
client = OpenAI()

# 1. Subir archivos
train = client.files.create(file=open("train.jsonl", "rb"), purpose="fine-
tune")
val   = client.files.create(file=open("val.jsonl", "rb"), purpose="fine-
tune")

# 2. Crear el trabajo (use un modelo que el proveedor liste como ajustable)
job = client.fine_tuning.jobs.create(
    training_file=train.id,
    validation_file=val.id,
    model="<modelo-ajustable-vigente>",
    hyperparameters={"n_epochs": 3},
)

# 3. Consultar estado
print(client.fine_tuning.jobs.retrieve(job.id).status)

# 4. Usar el modelo resultante
# client.chat.completions.create(model="ft:...", messages=[...])
```

Nota: los modelos disponibles para ajuste, los precios y los límites cambian; consulte la documentación oficial del proveedor antes de ejecutar.

B) Fine-tuning local con LoRA / QLoRA

Teoría de LoRA (*Low-Rank Adaptation*). En lugar de modificar la matriz de pesos original *W* (congelada), LoRA aprende dos matrices pequeñas *A* y *B* de rango bajo *r*, de modo que el cambio aprendido es $\Delta W = B \cdot A$. Solo se entrenan *A* y *B*, lo que reduce los parámetros entrenables a menos del 1 % del total en muchos casos.

Ventajas: requiere mucha menos memoria; entrena más rápido; el resultado es un **adaptador** liviano (decenas o cientos de MB) que se acopla al modelo base; se pueden tener varios adaptadores para un mismo modelo base.

QLoRA: carga el modelo base cuantizado a 4 bits y entrena adaptadores LoRA encima, lo que permite ajustar modelos de 7B–8B en una GPU de unos 12–16 GB de VRAM.

Hiperparámetros clave:

Parámetro	Significado	Valor orientativo
`r` (rango)	Capacidad del adaptador	8 – 32
`lora_alpha`	Escala del adaptador	16 – 64 (a menudo 2× `r`)
`lora_dropout`	Regularización	0,05
`learning_rate`	Velocidad de aprendizaje	1e-4 – 2e-4
`epochs`	Pasadas por el dataset	1 – 3
`batch size`	Ejemplos por paso	Según memoria disponible

Herramientas accesibles:

- **Hugging Face** (`transformers`, `peft`, `trl`, `bitsandbytes`): estándar de la industria.
- **Unsloth:** biblioteca que acelera y reduce la memoria del ajuste con LoRA/QLoRA; muy usada en Google Colab.
- **Google Colab** (GPU gratuita o de pago) o **Kaggle Notebooks**: permiten entrenar sin GPU propia.
- **Axolotl, LLaMA-Factory:** marcos configurables por archivo.
- **MLX** (Apple Silicon): ajuste local en Mac.

Esquema de entrenamiento con Hugging Face + PEFT + TRL (QLoRA):

```
import torch
from datasets import load_dataset
from transformers import AutoModelForCausalLM, AutoTokenizer,
BitsAndBytesConfig
from peft import LoraConfig
```

```
from trl import SFTTrainer, SFTConfig

MODELO = "<modelo-base-instruct>"   # por ejemplo, un Llama/Mistral/Qwen de
3B-8B

bnb = BitsAndBytesConfig(load_in_4bit=True,
                         bnb_4bit_quant_type="nf4",
                         bnb_4bit_compute_dtype=torch.bfloat16)

tokenizer = AutoTokenizer.from_pretrained(MODELO)
modelo = AutoModelForCausalLM.from_pretrained(MODELO,
                                              quantization_config=bnb,
                                              device_map="auto")

lora = LoraConfig(r=16, lora_alpha=32, lora_dropout=0.05,
                  target_modules="all-linear", task_type="CAUSAL_LM")

datos = load_dataset("json", data_files={"train": "train.jsonl",
                                         "validation": "val.jsonl"})

config = SFTConfig(output_dir="salida-lora",
                   num_train_epochs=2,
                   per_device_train_batch_size=2,
                   gradient_accumulation_steps=8,
                   learning_rate=2e-4,
                   logging_steps=10,
                   eval_strategy="steps", eval_steps=50,
                   save_strategy="epoch")

trainer = SFTTrainer(model=modelo, args=config, peft_config=lora,
                     train_dataset=datos["train"],
                     eval_dataset=datos["validation"],
                     processing_class=tokenizer)
trainer.train()
trainer.save_model("adaptador-lora")
```

Nota: las APIs de estas bibliotecas evolucionan con rapidez (nombres de argumentos, clases). Verifique la versión instalada y su documentación; en Colab, los cuadernos de ejemplo de Unsloth y de Hugging Face se mantienen actualizados.

Llevar el resultado a Ollama: se fusiona (*merge*) el adaptador con el modelo base, se convierte a formato GGUF con las herramientas de llama.cpp y se registra con un Modelfile (`FROM ./mi-modelo.gguf`). Alternativamente, Ollama permite aplicar un adaptador compatible con la instrucción `ADAPTER` en el Modelfile.

Señales de que el entrenamiento va bien o mal:

- *Pérdida (loss) de entrenamiento y validación ambas bajan:* bien.
- *Entrenamiento baja pero validación sube:* **sobreajuste**; reduzca épocas, aumente datos o disminuya `r`.
- *Ninguna baja:* revise la tasa de aprendizaje y el formato de los datos.
- *Olvido catastrófico:* el modelo mejora en la tarea pero pierde capacidades generales; mitigue con LoRA de bajo rango, pocas épocas y mezclando ejemplos generales.

10.2.4 Evaluación del modelo ajustado frente al modelo base

Teoría. Un modelo ajustado no es mejor por definición: hay que demostrarlo. La evaluación debe usar el **conjunto de prueba** que nunca vio el modelo durante el entrenamiento.

Tres líneas de evaluación complementarias:

1. **Métricas automáticas** (cuando la tarea tiene respuesta objetiva):
   - Clasificación: exactitud (*accuracy*), precisión, *recall* y F1.
   - Extracción estructurada: porcentaje de JSON válidos y coincidencia exacta de campos.
   - Texto generado: métricas como ROUGE o BLEU (limitadas), o similitud semántica con embeddings.
   - *Perplexity* sobre el conjunto de validación.
2. **Evaluación con rúbrica** (LLM como juez): un modelo potente califica las respuestas según criterios definidos (exactitud, tono, formato, seguridad). Debe calibrarse contra juicio humano, porque los modelos juez tienen sesgos (por ejemplo, preferir respuestas largas).
3. **Evaluación humana ciega:** evaluadores comparan respuestas de ambos modelos en parejas, sin saber cuál es cuál, y eligen la mejor (*win rate*).

Qué comparar siempre:

Dimensión	Pregunta
Calidad en la tarea objetivo	¿Mejor que el base y que el base con buen prompting?
Formato y consistencia	¿Respeta siempre el formato requerido?
Capacidades generales	¿Se degradaron? (prueba con tareas no relacionadas)
Seguridad y alucinaciones	¿Inventa más o menos datos? ¿Se niega cuando debe?

Dimensión	Pregunta
Latencia y costo	¿Cambió el tiempo de respuesta o el costo por consulta?

Línea base justa: compare el modelo ajustado no solo contra el modelo base "crudo", sino contra el modelo base con un **buen prompt y ejemplos** (few-shot). Si el prompting logra lo mismo, el fine-tuning no se justifica.

Esquema de comparación por pares (Python):

```
import json, ollama

def responder(modelo, mensajes):
    r = ollama.chat(model=modelo, messages=mensajes, options={"temperature":
0})
    return r["message"]["content"]

resultados = []
with open("test.jsonl", encoding="utf-8") as f:
    for linea in f:
        d = json.loads(linea)
        entrada = d["messages"][:-1]
        esperado = d["messages"][-1]["content"]
        resultados.append({
            "entrada": entrada[-1]["content"],
            "esperado": esperado,
            "base": responder("modelo-base", entrada),
            "ajustado": responder("modelo-ajustado", entrada),
        })

json.dump(resultados, open("comparacion.json", "w", encoding="utf-8"),
          ensure_ascii=False, indent=2)
```

10.2.5 Consideraciones éticas y de costos del fine-tuning

Consideraciones éticas y legales:

- **Privacidad y consentimiento:** no entrene con datos personales sin base legal y autorización. Anonimice. Un modelo puede *memorizar* y reproducir fragmentos de sus datos de entrenamiento.
- **Propiedad intelectual y licencias:** verifique que tiene derecho a usar los datos y que la licencia del modelo base permite el ajuste y el uso comercial. Revise los términos de uso si usa salidas de otros modelos como datos de entrenamiento.

- **Sesgos:** el ajuste puede amplificar sesgos presentes en los datos. Audite con casos de prueba que cubran diversos grupos y contextos.
- **Seguridad:** el fine-tuning puede debilitar las salvaguardas del modelo base. Pruebe con casos de uso indebido y mantenga filtros y supervisión humana.
- **Transparencia:** informe a los usuarios cuando interactúan con un sistema de IA y documente el modelo (qué datos, qué versión, qué limitaciones).
- **Responsabilidad:** defina quién responde por los errores del modelo y mantenga trazabilidad (versiones de datos, de modelo y de evaluación).
- **Impacto ambiental:** el entrenamiento consume energía; LoRA y modelos pequeños reducen la huella.

Consideraciones de costos:

Rubro	Qué considerar
Preparación del dataset	Suele ser el costo mayor: horas de curación y revisión humana
Entrenamiento	API: costo por tokens de entrenamiento × épocas. Local/Colab: horas de GPU
Inferencia	API: tarifa por token del modelo ajustado (a menudo superior a la del base). Local: hardware y energía
Evaluación	Tiempo humano y llamadas a un modelo juez
Mantenimiento	Reentrenar cuando cambien los datos, la tarea o el modelo base

Estimación en API: costo de entrenamiento ≈ (tokens del dataset × épocas) × precio por token de entrenamiento. Calcule primero con el dataset completo y con una prueba pequeña.

Buenas prácticas de costo: empezar con un dataset pequeño y de calidad; probar primero con un modelo pequeño; usar LoRA/QLoRA en lugar de ajuste completo; limitar épocas; evaluar temprano para detener entrenamientos que no mejoran; y confirmar que el beneficio supera el costo de las alternativas (prompting o RAG).

PRÁCTICAS DEL MÓDULO 10.2

PRÁCTICA 3 — Preparación de un dataset propio para fine-tuning (≈ 4 h)

Objetivo: construir un dataset limpio, anonimizado y validado para una tarea concreta.

Casos de uso sugeridos (elija uno):

- Asistente de soporte con el tono de una marca o institución.
- Clasificador de solicitudes o correos por categoría y prioridad.
- Extractor de datos de facturas o correos hacia JSON.
- Redactor de respuestas a preguntas frecuentes de un área (RRHH, ventas, atención al estudiante).

Pasos:

1. **Definir la tarea** en una frase y especificar el formato de salida esperado.
2. **Recolectar al menos 60 ejemplos** (idealmente 100 o más) a partir de material propio o simulado.
3. **Anonimizar:** reemplace datos personales por marcadores (`[NOMBRE]`, `[CORREO]`, `[TELÉFONO]`). Documente qué reglas aplicó.
4. **Redactar las respuestas ideales** y una guía de estilo breve (5 reglas) que todas deben cumplir.
5. **Incluir al menos 10 casos difíciles** (fuera de alcance, ambiguos, mal escritos).
6. **Convertir a JSONL** en formato de chat.
7. **Ejecutar el script de validación y división** de la sección 10.2.2 (80/10/10).
8. **Revisión cruzada:** otro participante revisa 15 ejemplos y marca inconsistencias.

Entregable: archivos `train.jsonl`, `val.jsonl`, `test.jsonl`; documento de 1 página con la definición de la tarea, la guía de estilo, las reglas de anonimización y el resumen de la revisión cruzada.

Rúbrica (20 puntos): definición de la tarea (3), calidad y consistencia de ejemplos (6), anonimización (4), casos difíciles (3), validación y división correctas (4).

PRÁCTICA 4 — Proceso de fine-tuning de un modelo con LoRA (≈ 6 h)

Objetivo: ajustar un modelo pequeño con LoRA/QLoRA usando el dataset de la Práctica 3 y obtener un adaptador funcional.

Entorno: Google Colab con GPU (o equipo propio con GPU compatible). Modelo base *instruct* de 1B–8B según la VRAM disponible.

Pasos:

1. **Preparar el entorno:** instale las bibliotecas (`transformers` , `peft` , `trl` , `datasets` , `bitsandbytes` , o use Unsloth) y cargue el modelo base cuantizado.
2. **Línea base:** antes de entrenar, genere respuestas del modelo base para 10 ejemplos del conjunto de prueba y guárdelas.
3. **Configurar LoRA:** `r=16` , `lora_alpha=32` , `dropout=0.05` , tasa de aprendizaje 2e-4, 2 épocas.
4. **Entrenar** y registre las curvas de pérdida de entrenamiento y validación.
5. **Experimentar:** repita con una segunda configuración (por ejemplo, `r=8` y 1 época, o 3 épocas) y compare las curvas. Identifique si hay sobreajuste.
6. **Probar el adaptador:** genere respuestas para los mismos 10 ejemplos y compárelas con la línea base.
7. **Exportar:** guarde el adaptador; opcionalmente fusione con el modelo base, convierta a GGUF y cargue en Ollama.

Entregable: cuaderno (notebook) ejecutado y reproducible, adaptador guardado, gráfico de pérdidas y un análisis de una página sobre las diferencias observadas entre las dos configuraciones.

Rúbrica (20 puntos): configuración correcta (5), entrenamiento completo con registro de métricas (5), experimento comparativo (5), análisis e interpretación (5).

PRÁCTICA 5 — Proyecto: modelo personalizado evaluado frente al modelo base (≈ 8 h)

Objetivo: integrar todo el nivel en un proyecto final que demuestre, con evidencia, si el modelo personalizado supera al modelo base.

Alcance del proyecto:

1. **Caso de uso y justificación:** describa el problema, por qué se eligió fine-tuning (frente a prompting y RAG) y por qué se ejecuta de forma local o en la nube.
2. **Modelos a evaluar (mínimo tres):** (a) modelo base sin instrucciones especiales, (b) modelo base con prompt optimizado y ejemplos, (c) modelo ajustado con LoRA (o con la API).
3. **Evaluación sobre el conjunto de prueba:**
   - Métricas automáticas pertinentes a la tarea.
   - Evaluación con rúbrica de 1 a 5 en tres criterios definidos por usted.
   - Comparación ciega por pares con al menos dos evaluadores humanos en 15 casos.
   - Prueba de capacidades generales con 5 preguntas ajenas a la tarea.
   - Prueba de seguridad con 5 casos de uso indebido o fuera de alcance.

4. **Costo y viabilidad:** tabla con el costo de preparación, entrenamiento e inferencia, y una proyección mensual para un volumen de consultas definido.
5. **Análisis ético:** riesgos de privacidad, sesgos, licencias y medidas de mitigación.
6. **Demostración:** el modelo funcionando (en Ollama, LM Studio o mediante la API) con un caso en vivo.

Entregables: informe técnico (6–10 páginas) con tablas y gráficos; repositorio o carpeta con dataset, código, adaptador y resultados; presentación de 10 minutos con demostración.

Estructura sugerida del informe:

1. Resumen ejecutivo
2. Caso de uso y decisión técnica
3. Dataset (origen, limpieza, anonimización, tamaño)
4. Método de ajuste y configuración
5. Resultados de la evaluación
6. Costos y viabilidad
7. Consideraciones éticas
8. Conclusión y recomendación ("¿se justifica el fine-tuning?")

Rúbrica (40 puntos): justificación del enfoque (5), calidad del dataset (5), ajuste técnico correcto (7), rigor de la evaluación (10), análisis de costos (4), análisis ético (4), demostración y comunicación (5).

EVALUACIÓN Y DISTRIBUCIÓN HORARIA DEL NIVEL

Componente	Horas	Peso en la calificación
Módulo 10.1 — Teoría y demostraciones	6	—
Prácticas P1 y P2	4	20 % (P1 8 %, P2 12 %)
Módulo 10.2 — Teoría y demostraciones	6	—
Prácticas P3 y P4	4	30 % (P3 12 %, P4 18 %)
P5 — Proyecto integrador	Trabajo guiado y autónomo	40 %

Componente	Horas	Peso en la calificación
Prueba escrita breve (conceptos clave)	—	10 %

Nota: el desglose por práctica es una propuesta que el instructor puede ajustar a su calendario; las horas de práctica se reparten dentro de las 10 h de cada módulo.

Glosario

- **Adaptador (LoRA):** conjunto pequeño de pesos entrenados que se acopla a un modelo base congelado.
- **Cuantización:** reducción de la precisión numérica de los pesos para ahorrar memoria y acelerar la inferencia.
- **Época:** una pasada completa por el dataset de entrenamiento.
- **Fine-tuning:** continuación del entrenamiento de un modelo preentrenado con datos propios.
- **GGUF:** formato de archivo de modelos cuantizados usado por llama.cpp, Ollama y LM Studio.
- **Instruct:** variante de un modelo ajustada para seguir instrucciones.
- **JSONL:** archivo de texto con un objeto JSON por línea.
- **LoRA / QLoRA:** técnicas de ajuste eficiente en parámetros (QLoRA parte de un modelo cuantizado a 4 bits).
- **Olvido catastrófico:** pérdida de capacidades generales tras un ajuste excesivo.
- **Open-weights:** modelo con pesos descargables, no necesariamente con licencia totalmente abierta.
- **PEFT:** *Parameter-Efficient Fine-Tuning*.
- **RAG:** generación aumentada por recuperación de documentos.
- **SFT:** *Supervised Fine-Tuning*, ajuste con pares entrada-salida.
- **Sobreajuste:** el modelo memoriza el entrenamiento y generaliza mal.
- **Ventana de contexto:** máximo de tokens que el modelo procesa a la vez.

Referencias y recursos recomendados

- Documentación oficial de Ollama y LM Studio.
- Hugging Face: documentación de `transformers`, `peft`, `trl` y cursos gratuitos.
- Documentación de Unsloth y de llama.cpp.

- Documentación de fine-tuning del proveedor de API elegido (modelos disponibles y precios vigentes).
- Artículos originales: *LoRA: Low-Rank Adaptation of Large Language Models* (Hu et al., 2021) y *QLoRA: Efficient Finetuning of Quantized LLMs* (Dettmers et al., 2023).